```scala
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(a) VDotProducts 乘积计算

```scala
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

(b) VDotReduce 归约求和